



Installing the SU package

Remember that the U in SU stands for “Unix”. Thus, you must be running some form of a Unix or Unix-like environment to use SU. Today, many choices are available. There are, of course, many variations of Linux, each which works equally well as a platform for SU. Another Unix option is Mac OSX, which was based on FREE BSD Unix.

In more recent releases of Microsoft Windows, there is a “bash shell” environment available, which is a version of Ubuntu Linux. An older solution to the Windows versus Unix question is a system called Cygwin, which comes in 32 bit and 64 bit systems. For Microsoft Windows installations, you will need to also add an X-windows server. Finally, users may configure their windows as dual-boot platforms, enjoying either a completely linux or a completely Microsoft environment.

More recently in Ubuntu 20.04 and later, the Linux environment for Window is called the Windows Linux Subsystem (WSL) and is an effective environment for installing Linux operating systems. Yet another tool is the Hyper-V virtual machine environment.

Everything that follows in this discussion is executed on the “Unix command line”. To do anything further requires that you open your systems “Terminal” application so that you have access to the command line. This can be a culture shock for those who grew up in the “land of point and click drag and drop” and “there's an APP for that.” This may seem pretty old school, but, in reality this is the “expert user” environment.

Preliminaries

To install SU, you have to compile it on your computer. This means that you will need more than just the basic installation of your operating system.

Install developers' libraries

You will need that system's equivalent of a “developers” package.

For Linux-based systems you will need to install many “dev” packages that are offered with, but not included by default. These include the C compiler (usually some version of GCC) and the X-windows graphics libraries. There are Fortran codes in SU, however, these are not essential to the package. On Mac OS X systems, there is a developer's environment called Xcode that needs to be installed. There is also an X-windows environment called X-Quartz that is necessary for the X-windows graphics packages

Download and unbundle the current SU release

Nomenclature point. What we refer to as a “directory” in all subsequent discussions is the same thing as a “folder”.

Select a convenient location on your system and make it writeable by yourself as a regular user. For this discussion I will assume that the convenient location is /usr/local/cwp and that the regular user is named 'john'. The commands that would be used to make this directory are

```
$ sudo mkdir /usr/local
```

```
$ sudo mkdir /usr/local/cwp
```

```
$ chown john /usr/local/cwp
```

(Note that the leading \$ on each line represents

For all future reference this directory is your CWPROOT directory.

You can download the latest release of SU [here \[https://nextcloud.seismic-unix.org/index.php/s/LZpzc8jMzbWG9BZ\]](https://nextcloud.seismic-unix.org/index.php/s/LZpzc8jMzbWG9BZ).

Move the current distribution that has a name like cwp_su_all_4XRX.X.tgz where the Xs are numbers and the .tgz extension means that the file is a gzipped Unix tar compressed and bundled archive—often called “tarball.” Please move the tar archive to the directory you have created. In my example, regular user “john” can do all of the subsequent steps, without ever having to become the “super user”.

The commands for unbundling the tar archive are:

```
$ gunzip cwp_su_all_4XRX.X.tgz
```

```
$ tar -xvf cwp_su_all_4XRX.X.tar
```

A tree of directories with “src” (pronounced “source”) will appear in the CWPROOT directory. This src directory contains the full source code for SU.

Setting your CWPROOT path

To permit the installation process to find the source code, You will need to set the CWPROOT environment variable. To do this, you will need to know what your working shell environment is.

Type:

```
$ echo $SHELL
```

Note that the first dollar sign merely represents the command line prompt, which may be different on your system. The object \$SHELL is read as “value of” the variable SHELL. Here SHELL is an environment variable, that has the value of /bin/sh, /bin/bash, /bin/ksh, /bin/zsh, /bin/csh, /bin/tcsh and it is customary to write environment variables all in capital letters.

Shell environment: /bin/bash (the Bourne Again Shell)

The most common shell environment these days is /bin/bash and the configuration files in your home directory are .bash and .bash_profile These files must be edited so that the environment variable CWPROOT and that \$CWPROOT/bin are on the users' path.

To do this, type:

```
$ cd
```

If you type

```
$ pwd
```

you will be in your home directory, which for our sample user “john” is /Users/john If you type:

```
$ ls -ls
```

you will see a bunch of files and directories whose names are preceded by the symbol “.”

Using your favorite editor add the following lines to the bottom of the files .bashrc and .bash_profile Create these files if they do not already exist. Note that our user john has a CWPROOT path of /usr/local/cwp. You need to set this to the directory path name that you used on your system. Take care that the dots “.” are there

```
# set the CWPROOT variable
```

```
export $CWPROOT=/usr/local/cwp
```

```
# set the PATH variable
```

```
export PATH=$PATH:$CWPROOT/bin.
```

Shell environments: /bin/ksh (Korn Shell) and /bin/sh (Bourne Shell)

The configuration file for these environments is called `.profile` however the syntax is the same, so add the same lines to the bottom of `.profile`.

Caveat: on most modern systems `/bin/sh` is merely a link to the `/bin/bash` shell, so the configuration file will be `.bashrc` and `.bash_rc`.

Shell environment: `/bin/zsh` (Z-shell)

This is an important environment, because MacOSX Catalina and later releases make this the default shell for new installations.

The configuration file for these Z-shell is called `.zshrc` however the syntax is the same, so add the same lines to the bottom of `.zshrc`

Shell environments: `/bin/csh` (C-shell) and `/bin/tcsh` (T-shell)

The C-Shell is a totally different environment. These are less commonly used environments, today. The configuration file is called `.cshrc` again, do not forget the dot “.” at the beginning of the filename

To make the equivalent changes: Add the line at the top of the file:

```
setenv CWPROOT /usr/local/cwp
```

```
set path=( $path $CWPROOT/bin. )
```

(again remember that you should use your CWPROOT path instead of `/usr/local/cwp` if you have chosen something different.

Implementing the changes

Open a new terminal window and type:

```
. .bashrc
```

or

```
. .profile
```

or

```
source .cshrc
```

depending on which SHELL environment that you are working under.

Then type:

```
$ echo $CWPROOT
```

You should get the value of CWPROOT

```
$ echo $PATH
```

or

```
$ echo $path
```

You should see your CWPROOT path with bin appended

If not you may have made an error, and in your other terminal window edit the appropriate file for your shell environment and repeat.

If, however, everything looks fine then log out all the way to the system's login panel and log back in. You are ready to begin installing SU.

Setting the Makefile.config

Now you are ready to begin installing SU. Note that if you get error messages along the way, these are important clues to adding or changing items that are necessary for the SU package.

```
$ cd $CWPROOT/src
```

In the directory `$CWPROOT/src/configs` there are a collection of Makefile configuration files. Select the one that is most appropriate for your system

```
$ cd $CWPROOT/src/configs
```

```
$ cp Makefile.config_for_your_system $CWPROOT/src/Makefile.config
```

```
$ cd $CWPROOT/src/
```

Bring your new Makefile.config into your favorite editor. If you are using Fedora Linux, you may need to change the line

```
XDRFLAG = -DSUXDR
```

to

```
XDRFLAG =
```

The Fedora Project has branched the SUN libraries off into a separate location. If you have been using SU for a long time and have files in the XDR format, and want to continue using the XDR format, then you will have to install the rpc libraries and make changes to the Makefiles, to get SU to install with XDR.

Compiling the Codes

To begin the compilation process:

```
$ cd $CWPROOT/src
```

```
$ make install
```

You will be asked to accept a license and given the option of sending an installation message back to the originator of SU. You are free to not send the message if you do not want to.

When this completes, type:

```
$ make xtinstall
```

Should this also complete with no error messages, then most of what you would need of SU is installed.

Additional items

There are additional items to install for the complete package. These are

`make xminstall` — for FFTLAB, a fun educational app for exploring the Fourier Transform

`make mglinstall` — Open GL applications, Graphics codes that could be built upon.

`make finstall` — Fortran codes, modeling and anisotropy codes.

`make sinstall` — SFIO installation for segdread (the SEG D to SU data reader)

None of these is essential for running most of SU.

Installing SU on specific platforms

Ubuntu 22.04

After you have installed Ubuntu 22.04, no matter what environment you are using for this, there are addition packages that must be installed.

```
$ sudo apt-get install xorg
```

```
$ sudo apt-get update
```

```
$ sudo apt-get install xorg
```

```
$ sudo apt-get install gcc
```

```
$ sudo apt-get install libx11-dev
```

```
$ sudo apt-get install libxt-dev
```

```
$ sudo apt-get install libxext-dev
```

```
$ sudo apt-get install make
```

```
$ sudo apt-get install libmotif-dev
```

```
$ sudo apt-get install libtirpc-dev
```

Case study: Linux Mint notes contributed by a user, including comments on the sfinstall portion

```
# setup base dir sudo mkdir -p /opt/cwp/44R19 cd /opt/cwp # Replace USER below with your username sudo chown USER:USER 44R19
```

```
# Add env vars to bashrc. cat » ~/.bashrc « eof # Seismic Unix export CWPROOT=/opt/cwp/44R19 export PATH=$PATH:/opt/cwp/44R19 eof
```

```
# Check CWPROOT is set.
```

```
. ~/.bashrc
```

```
cd $CWPROOT
```

```
# grab and unpack SU
```

```
cp ~/Downloads/cwp_su_all_44R19.tgz .
```

```
tar -xzf cwp_su_all_44R19.tgz
```

```
# set the make config to Linux_x86_64.
```

```
cd src
```

```
cp configs/Makefile.config_Linux_x86_64 Makefile.config
```

```
# Make the base seismic unix.
```

```
make install
```

```
# Install the required libraries for the X-toolkit windows.
```

```
sudo apt-get install libx11-dev libxt-dev
```

```
make xinstall
```

```
# Install the required libraries for the Fortran codes
```

```
sudo apt-get install gfortran
```

```
# add the string '-std=legacy' to the FFLAGS line of the Makefile.config (line 97 for me)
```

make finstall

Install Mesa/OpenGL items

sudo apt-get install libglu1-mesa-dev freeglut3-dev libxmu-dev libxmu-headers libxi-dev

make mglinstall

Build Utils make utils

Install libraries and build motif based applications sudo apt-get install aptibmotif-dev make xminstall

Edit code and build SFIO version of SEGDREAD # remove lines 399 to 404 of src/Sfio/src/lib/sfio/Stdio_b/Sfstdio.c

```
printf("/* Linux7.2 requires __FILE in wchar.h - we fake it here */\n");
printf("#include          \"FEATURE/sfio\"\n");
printf("#if _typ__FILE\n");
printf("typedef FILE          *__FILE;\n");
printf("#endif\n");
```

Copy

remove lines 5 to 10 of src/Sfio/src/lib/sfio/Stdio_s/stdio_s.h

```
#include          "FEATURE/sfio"
```

```
#if _typ__FILE /* Redhat7.3 requires __FILE in wchar.h */
typedef struct _sfio_s  *__FILE;
```

```
#endif
```

Copy

make sfinstall